

Efficient Query Rewrite Rule Discovery via Standardized Enumeration and Learning-to-Rank

Yuan Zhang*, Yuxing Chen[†] ✉, Yuekun Yu*, Jinbin Huang*, Rui Mao* ✉,
Anqun Pan[†], Lixiong Zheng[†], Jianbin Qin* ✉

*Shenzhen Institute of Computing Science, Shenzhen University, Shenzhen, China
Email: {2300271049,2410104038}@email.szu.edu.cn, {jbhuang,mao,qinjianbin}@szu.edu.cn

[†]Tencent Inc., Shenzhen, China
Email: {axingguchen,aaronpan,paterzheng}@tencent.com

Abstract—Query rewriting is essential for database performance optimization, but existing automated rule enumeration methods suffer from exponential search spaces, severe redundancy, and poor scalability, especially when handling complex query plans with five or more nodes, where a *node* represents an operator in the plan tree. We present SLER, a scalable system that enables efficient and effective rewrite rule discovery by combining standardized template enumeration with a learning-to-rank approach. SLER uses *standardized templates*, abstractions of query plans with operator structures preserved but data-specific details removed, to eliminate structural redundancies and drastically reduce the search space. A learn-to-rank model guides enumeration by pre-filtering the most promising template pairs, enabling scalable rule generation for large-node templates.

Evaluated on over 11,000 real-world SQL queries from both open-source and commercial workloads, SLER has automatically constructed a rewrite rule repository exceeding 1 million rules—the largest empirically validated rewrite rule library to date. Notably, at the scale of one million rules, SLER supports query plan templates with complexity up to *channel-level* depth. This unprecedented scale opens the door to discovering highly intricate transformations across diverse query patterns. Critically, SLER’s template-driven design and learned ranking mechanism are inherently extensible, allowing seamless integration of new and complex operators, paving the way for next-generation optimizers powered by comprehensive, adaptive rule spaces.

I. INTRODUCTION

Query rewriting is a critical technique for enhancing database performance [1]–[4], optimizing queries before they reach the internal engine [5]–[7]. Traditional manual rules are unsustainable for complex workloads [6]–[13]. To address this, a general framework is to discover rules in advance and performs SQL-to-SQL rewrites before queries reach the database engine. This ensures discovery is a one-time cost while the query-time application remains efficient. By identifying intricate structural patterns that simple rules often miss, this external approach often provides a portable solution for any SQL-compliant DBMS without requiring internal kernel modifications or system-specific adaptations.

Challenges. Existing SOTA works like WeTune on query rewrite rule exploration exhibit limitations in both efficiency and effectiveness due to two key deficits: (1) a lack of effective rules for specific complex patterns, resulting in failures to simplify intricate queries (**ineffectiveness**); (2) the need for multiple rule-identifying iterations to optimize queries with

more than four nodes, where a node represents a specific operator (e.g., Join or Project) in the query plan tree (**inefficiency**), coupled with its constrained enumeration space, defined as the combinatorial set of all possible operator arrangements and structures, which prevents reduction to the simplest form. Examples of these limitations are provided below.

For example, WeTune [14] results in prohibitively high complexity (4320 CPU hours for ≤ 4 nodes) and poor quality (over 90% redundant; 50% trivial). Scaling is fundamentally infeasible, with 5-node rules requiring approximately six months and 6-node rules requiring a decade or more. This limitation is critical because real-world workloads [15], [16] and production databases (like PostgreSQL [17]) frequently involve queries with more than four nodes, rendering WeTune’s rule base insufficient for practical optimization scenarios [18].

Example 1. Table 1 showcases three original queries (more detailed description in §II), alongside their rewritten versions generated by WeTune and their corresponding ideal forms. The ideal form, defined as the syntactically simplest representation, typically exhibits superior performance in database systems relative to the original query structure. WeTune fails to rewrite query q_1 due to the absence of applicable transformation rules in its rule base. Although WeTune’s rewritten form of query q_5 aligns with the ideal solution, the computational overhead incurred during rewriting is notably higher. For query q_7 , WeTune can only produce a non-minimal form. This stems from its constrained rule base, which is primarily designed for query patterns with four or fewer nodes.

Additionally, assessing rule effectiveness using formal methods like SMT solvers (e.g., Z3 [19]) is constrained by unresolved complexity questions such as NP=coNP [20]–[22], making proof lengths unreliable metrics for rule quality, especially when query plans are with larger node sizes. Enumerating rules for templates with >6 nodes faces combinatorial explosion, as 99% of time (couple of years [14]) is consumed by verification calls, rendering existing methods impractical.

Contributions. This paper proposes SLER, a query rewrite framework with a rule enumerator, deduplicator, and ranker, which efficiently identifies effective rules that can improve database system performance, addressing the key challenges of efficiency and effectiveness in query rewrite rule identification.

TABLE I: Example queries and their rewritten forms by WeTune [14] versus our method (SLER). WeTune fails to rewrite q_1 , transforms q_7 into a non-minimal form, and incurs higher rewriting overhead for q_5 .

Original Query	WeTune	Ideal (SLER)
q_1 : <code>SELECT e1.dept_id AS s2 FROM (SELECT dept_id AS s0, region FROM Employees WHERE salary > 50000) e1 LEFT JOIN (SELECT region AS s1, dept_id FROM Employees) e2 ON e1.region = e2.region</code>	q_2 : <code>SELECT e1.dept_id AS s2 FROM (SELECT dept_id AS s0, region FROM Employees WHERE salary > 50000) e1 LEFT JOIN (SELECT region AS s1, dept_id FROM Employees) e2 ON e1.region = e2.region</code>	q_3 : <code>SELECT dept_id AS s2 FROM Employees WHERE salary > 50000</code>
q_4 : <code>SELECT DISTINCT s.Region, s.TotalSales FROM Sales s WHERE s.Category = 'Electronics' AND s.Region IN (SELECT n.Region FROM Sales n WHERE n.Category = 'Electronics')</code>	q_5 : <code>SELECT DISTINCT s.Region, s.TotalSales FROM Sales s WHERE Category = 'Electronics'</code>	q_6 : <code>SELECT DISTINCT s.Region, s.TotalSales FROM Sales s WHERE Category = 'Electronics'</code>
q_7 : <code>SELECT e1.emp_id, e1.name, e1.dept_id FROM Employees e1 INNER JOIN (SELECT DISTINCT dept_id AS s0 FROM Projects) p ON e1.dept_id = p.dept_id WHERE e1.dept_id IN (SELECT e2.dept_id AS s1 FROM Employees e2 LEFT JOIN Projects p2 ON e2.dept_id = p2.dept_id)</code>	q_8 : <code>SELECT e1.emp_id, e1.name, e1.dept_id FROM Employees e1 INNER JOIN (SELECT DISTINCT e2.dept_id AS s0 FROM Projects p INNER JOIN Employees e2 ON p.dept_id = e2.dept_id) p ON e1.dept_id = p.dept_id</code>	q_9 : <code>SELECT e.emp_id, e.name, e.dept_id, p.dept_id AS s0 FROM Employees e LEFT JOIN (SELECT DISTINCT dept_id AS s0 FROM Projects) p ON e.dept_id = p.dept_id</code>

(1) **Efficiency.** The compositional *small-to-large* hypothesis, which attempts to derive complex rules by composing simpler ones, is empirically ineffective: only 9% of 4→2 node rules can be composed. This necessitates direct enumeration. To ensure both efficiency and full coverage, our system integrates three mechanisms. First, the enumerator (§IV-A) uses standardized plan templates for focused rule generation. Second, the ranker (§V), employing a LambdaMART model [23], [24], provides two operational modes: it can either pre-filter the top- k plan templates during enumeration to avoid exponential verification calls, enabling scalable discovery of multi-node rules (e.g., $n \geq 7$) at the cost of rule coverage; or it can be bypassed entirely during enumeration to ensure full coverage, and then applied post-enumeration to rank the final complete rule set. Finally, the deduplicator (§IV-B) implements the Reduced by Template Pair (RTP) algorithm, which prunes redundant rules by verifying validity and checking for redundancy among checksum-equivalent rules, thereby significantly reducing the final rule set and enumeration overhead.

(2) **Effectiveness.** SMT-based proof systems (e.g., Z3 [19]) struggle to reliably measure rule complexity due to unresolved theoretical issues such as NP=coNP, hindering effective rule prioritization. Our ranker (§V) addresses this by focusing on validating rule sets based on predicted effectiveness, rather than exhaustive proof. The model, trained on real-world SQL queries, uses features like operator types and expression complexities to predict rule scores. This allows it to sort rules in descending order of predicted performance impact. Crucially, this ranker efficiently handles both small-node and large-node queries (previously inaccessible), resolving scalability issues and expanding analyzable query patterns through integrated validity screening and deduplication techniques.

We summarize our contributions as follows:

- Formalize the “small-to-large” hypothesis and propose a standardized rule enumeration algorithm to reduce enumeration overhead and improve node coverage while balancing rule effectiveness (§IV-A).
- Develop new rule-based redundancy elimination optimization algorithms, incorporating normalization, regularization,

and structural similarity for efficient processing and candidate set reduction (§IV-B).

- Introduce a machine learning ranker to mitigate limitations in prioritizing exponential rules, thus boosting rule effectiveness and selectively reducing enumeration overhead (§V).
- Evaluate SLER on open-source and commercial queries, demonstrating its efficiency and effectiveness superiority over the SOTA WeTune (§VI).

II. RELATED WORK

Query optimization remains a cornerstone challenge in database systems, encompassing cardinality estimation (e.g., [25]–[29]), cost estimation (e.g., [30]–[34]), and query plan search (e.g., [13], [35]–[42]). These areas collectively strive to enhance query efficiency [26], [27], [43], yet gaps persist in handling complex workloads.

Recent research has investigated query rewriting rules using Large Language Models (LLMs), leveraging the models’ common knowledge, semantic understanding, and reasoning capabilities to improve generalization for complex patterns and detect redundancy. Key approaches include: QueryBooster [44], a non-invasive middleware service supporting user-defined rules; Liu et al.’s [45] use of NLR2s and anti-examples to reduce verification effort; R-Bot [46], a systematic LLM system with hybrid retrieval for robust, guaranteed rewriting; LLM-R2 [18], which enhances rule selection via LLM recommendations and contrastive learning optimization; Song et al.’s [47] training-free LLM agent system, expanding rewrite scope beyond fixed-rule expressiveness. However, these LLM-based methods often refine existing rules or evidence bases (hindering automatic discovery of entirely new rules), suffer from hallucinations and instability, and require high expertise for custom rule setup [44], creating a significant entry barrier. Notably, these approaches are orthogonal to the query rewrite explored in this paper and can be applied subsequently.

Query rewriting constitutes a pivotal technique for improving the performance of database-driven applications [1], as it optimizes poorly constructed queries before processing [5]–[7]. Traditional approaches rely on manually derived rules,

which are time-intensive and unsustainable. Despite reliance on extensive rule sets in production systems (e.g., Oracle [48], DB2 [49], TDSQL [4], [50], SQL Server [51], Amazon RDS [52]), many queries remain unoptimized, suggesting a significant gap of undiscovered effective rewrite rules.

Research focusing on tailored rule subsets [53] or application order [54] is limited, as they cannot expand the query plan search space or propose new rules. Beyond rule-based methods, synthesis-based techniques have emerged to explore rule-free optimization. For instance, Sia [55] utilizes synthesis for predicate optimization but is restricted to local modifications, while others focus on template-dependent recursive queries [56]. More recently, SlabCity [57] performs whole-query optimization via inductive synthesis. However, these methods are typically computationally intensive for individual queries and lack rule reusability, requiring expensive per-query processing. The current SOTA, WeTune [14], automates rule generation but faces critical limitations, including expansive enumeration spaces, inefficient processing, and pervasive redundant rules. To address, this paper aims to improve the efficiency and effectiveness of query rewrite rule enumeration.

Research Gaps Firstly, traditional query rewrite rules, reliant on programmer intuition, are inherently incomplete—especially inadequate for auto-generated SQL (from AI tools [58] or ORM frameworks) with counterintuitive structures. For instance, Table I shows three representative ORM-generated queries: q_1q_4 with redundant subqueries/self-joins; and q_7 introduces superfluous subqueries and joins. Notably, the nested structure of q_1 cannot be simplified by SOTA rules, despite its obvious inefficiency.

Secondly, Techniques like WeTune [14] are constrained to ≤ 4 -node queries, insufficient for real-world databases (e.g., PostgreSQL [17]) handling 5–8+ node queries. Additionally, WeTune’s exhaustive search is computationally prohibitive: ≤ 4 -node rule generation takes over 9 days (Table II), and scaling to 5–6 nodes would take months/decades—with a *small-to-large* mitigation strategy proving ineffective (§III-B).

Thirdly, Formal methods (e.g., Z3 [19]) cannot reliably evaluate rule effectiveness due to unpredictable proof lengths (linked to NP=coNP [20]–[22]). This lack of filtering clogs optimizers with trivial rules, increasing overhead—highlighting the need for scalable complex rule enumeration/prioritization.

III. PRELIMINARY AND OVERVIEW

This section introduces key concepts and formal definitions, followed by the motivation and overall design of this paper.

A. Preliminary

A query plan template (T) is defined as a directed tree, where each node corresponds to a relational algebra operator. Except for the Input operator, each operator takes one or two relations as input, executes algebraic computations via its semantics, and outputs a single relation. In line with WeTune [14], the operators supported in this work are as follows: (1) **Input(R)**: Represents an input relation, parameterized by a symbolic relation R (e.g., a table or subquery result). (2)

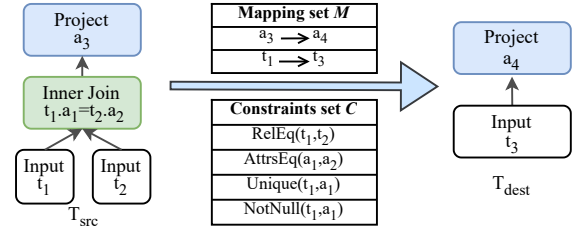


Fig. 1: Example of a rule model.

Project(a): Projects tuples to retain only the attributes specified in the list a (e.g., selecting specific columns from a table). (3) **Filter(p, a)**: Filters out tuples that do not satisfy predicate p , where p is evaluated using attributes in the list a (e.g., retaining rows where $age > 30$). (4) **InnerJoin($R_l.a_l, R_r.a_r$)**: Performs an inner join between the relations R_l (left) and R_r (right), retaining tuples where attributes $R_l.a_l$ and $R_r.a_r$ match. (5) **LeftJoin($R_l.a_l, R_r.a_r$)**: Performs a left join between the relations R_l and R_r , retaining tuples where attributes $R_l.a_l$ and $R_r.a_r$ match. (6) **RightJoin($R_l.a_l, R_r.a_r$)**: symmetric to LeftJoin (7) **InSub($R_l.a_l, R_r.a_r$)**: Filters tuples from R_l to retain only those where the value in $R_l.a_l$ exists in $R_r.a_r$ (e.g., where id in (select id from ...)). (8) **Distinct(a)**: Projects attributes a and removes duplicate tuples (equivalent to SQL’s DISTINCT keyword).

Constraints in constraint set C ensure that T_{src} and T_{dest} are semantic equivalence. They define relationships M between symbols (relations, attributes, predicates) in T_{src} and T_{dest} : (1) **RelEq(R_1, R_2)**: Relations R_1 and R_2 contain the identical tuples (i.e., they are logically the same dataset). (2) **AttrsEq(a_1, a_2)**: Attribute lists a_1 and a_2 are identical (e.g., both list $[id, name]$). (3) **PredEq(p_1, p_2)**: Predicates p_1 and p_2 are logically equivalent, i.e., $p_1 \Leftrightarrow p_2$ (e.g., $p_1 = (x \geq 5)$ and $p_2 = (x \geq 5)$ for integer x). (4) **SubAttrs(a_1, a_2)**: Each attribute in a_1 is also in a_2 (e.g., $a_1 = [id]$ and $a_2 = [id, name]$). (5) **RefAttrs(R_1, a_1, R_2, a_2)**: All values in R_1 on a_1 are also in R_2 on a_2 (e.g., all user IDs in R_1 exist in R_2). (6) **Unique(R, a)**: Values in R on a are unique (e.g., a primary key a). (7) **NotNull(R, a)**: No value in R on a is NULL (e.g., a column with a “NOT NULL” constraint).

A **query rewrite rule** formalizes how a source template can be transformed into a semantically equivalent destination template under specific constraints:

Definition 1 (Query Rewrite Rule). A query rewrite rule is $R = (T_{src}, T_{dest}, M, C)$, where T_{src}, T_{dest} are the source and destination templates, respectively; $M: V_{src} \rightarrow V_{dest}$ is a mapping associating nodes in T_{src} to T_{dest} ; C is a set of constraints that guarantee T_{src} and T_{dest} are semantically equivalent.

A rule of R is valid if two conditions are met: (1) T_{dest} contains fewer nodes than that T_{src} ; (2) C is minimal in enabling the equivalence between T_{src} and T_{dest} . Formally, C must be the most relaxed set of constraints such that removing any single constraint from C would break the semantic equivalence of the T_{src} and T_{dest} .

If both conditions hold, T_{src} is replaced with T_{dest} to form

TABLE II: Example of redundant rules: SLER retains only R_3 while WeTune generates all three, with two redundant.

Rule	Source Plan Template	Destination Plan Template	Extra Constraints
R_1	$\text{Proj}_{t_0.a_0}(\text{IJoin}_{t_0.a_1,t_0.a_0}(\text{Proj}_{t_0.a_0}(t_0), \text{Dedup}(\text{Proj}_{t_0.a_1}(t_0))))$	$\text{Proj}_{t_0.a_0}(t_0)$	$\text{NotNull}(t_0, a_0); \text{Unique}(t_0, a_1)$
R_2	$\text{Proj}_{t_0.a_1}(\text{IJoin}_{t_0.a_0,t_0.a_1}(\text{Proj}_{t_0.a_0}(t_0), \text{Dedup}(\text{Proj}_{t_0.a_1}(t_0))))$	$\text{Proj}_{t_0.a_0}(t_0)$	$\text{NotNull}(t_0, a_1); \text{Unique}(t_0, a_1)$
R_3	$\text{Proj}_{t_0.a_0}(\text{IJoin}_{t_0.a_0,t_0.a_1}(\text{Proj}_{t_0.a_0}(t_0), \text{Dedup}(\text{Proj}_{t_0.a_1}(t_0))))$	$\text{Proj}_{t_0.a_0}(t_0)$	$\text{NotNull}(t_0, a_0); \text{NotNull}(t_0, a_1); \text{Unique}(t_0, a_1)$

TABLE III: Summary of Key Notations.

Notation	Description
n	Node count (template size)
T_{src}, T_{dest}	Source and destination templates
C	Validity constraints (schema, predicates)
M	Structural mappings of operators
r	Verified rule tuple: $(T_{src}, T_{dest}, C, M)$
k	Top- k filtering threshold in LTR

an optimized query. The mapping M formalizes the transformation applied by rule R . For instance, Figure 1 illustrates a sample rule where a join operation in T_{src} is eliminated in T_{dest} under specific constraints (e.g., uniqueness of a join attribute). In this case, the mapping M means that the relation t_3 in T_{src} is substituted with t_1 , attribute a_4 is replaced by a_3 , and M is omitted from R sometimes, to improve readability. In summary, the primary notations are shown in Table III.

B. Motivation

To derive a not only valid but also effective rule, the current methods discover potential rewrite rules through the following processes. First, all possible plan templates are enumerated. To keep the search space manageable, the size of these templates is constrained, i.e., the number of operators in each template is limited to a small threshold. Next, all possible constraints are generated for each pair of enumerated plan templates. Finally, the most promising rules, that are the highest potential to improve query performance, are selected. To reduce the computational cost of enumerating complex rules, an intuitive approach is to explore a compositional method that infers from larger nodes to smaller ones step by step, which we formalize as the small-to-large hypothesis.

Definition 2 (Small-to-Large Composition). Let $\mathcal{R}_{n \rightarrow m}$ denote the set of query rewrite rules transforming an n -node plan to an m -node plan ($m < n$). For $m < k < n$, a composition operator $\circ : \mathcal{R}_{n \rightarrow k} \times \mathcal{R}_{k \rightarrow m} \rightarrow \mathcal{R}_{n \rightarrow m}$ is defined such that $R_{n \rightarrow m} = R_{n \rightarrow k} \circ R_{k \rightarrow m}$ if applying $R_{n \rightarrow k}$ followed by $R_{k \rightarrow m}$ yields $R_{n \rightarrow m}$. The small-to-large hypothesis holds if $|\mathcal{R}_{n \rightarrow m}| = |\mathcal{R}_{k \rightarrow m}|$ and $\mathcal{R}_{n \rightarrow m} \subseteq \{R_{n \rightarrow k} \circ R_{k \rightarrow m} \mid R_{n \rightarrow k} \in \mathcal{R}_{n \rightarrow k}, R_{k \rightarrow m} \in \mathcal{R}_{k \rightarrow m}\}$.

Empirically, this hypothesis often fails. For example, only 9% of rules in $\mathcal{R}_{4 \rightarrow 2}$ can be composed from $\mathcal{R}_{4 \rightarrow 3}$ and $\mathcal{R}_{3 \rightarrow 2}$. This indicates most complex rules require direct enumeration rather than composition. WeTune addresses this with brute-force enumeration but struggles to verify rules with five or more nodes. The root cause lies in WeTune’s inevitable generation of redundant rules. As shown in Table II, Rules R_1 and R_2 are redundant relative to Rule R_3 . Despite differences in join conditions or projected attributes in their source templates, they produce semantically equivalent results under unique and

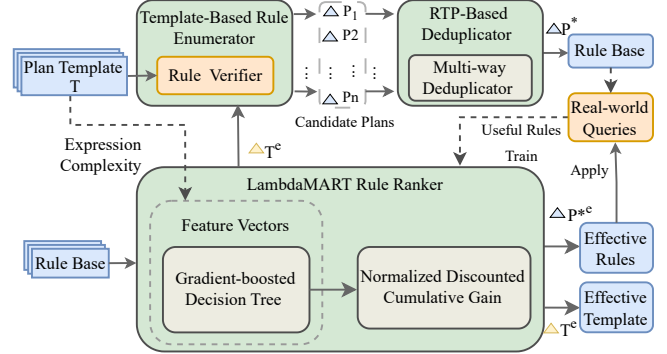


Fig. 2: System architecture of SLER.

non-null constraints. Yet WeTune generates all three rules exhaustively while our standardized enumeration selectively retains only R_3 (later in §IV-A).

Rule verification for semantic equivalence is a major challenge, where the complexity is formally captured by Proof Length and Optimality (defined and explained in the extended version [59]). Since optimal proof systems are theoretical (related to $\text{NP}=\text{coNP}$ [20]–[22]), WeTune’s exhaustive verification leads to high overhead. SLER leverages rule pruning to reduce verification of redundant rules to polynomial time for $n \leq 6$ nodes (§IV-B), a significant improvement. However, efficiency still degrades for $n \geq 7$. To enable enumeration beyond six nodes, SLER (§ V) uses a LambdaMART ranking model [23], [24] to prioritize effective rules. By ranking and filtering template pairs, we trade off rule completeness for enumeration speed, thereby enabling the scalable discovery of rules with $n \geq 7$ nodes. LambdaMART is chosen due to its inherent optimization for ranking tasks and its flexibility in handling complex features.

C. System Overview

SLER is designed to identify effective rules for improving the performance of real-world queries, as illustrated in Figure 2. Its workflow proceeds as follows: First, candidate standardized rules, explicitly designed to be non-redundant, are generated for each template, where these rules are fed into a rule equivalence checker to construct a standardized rule base. Subsequently, redundant rules are further Reduced by Template Pair (denoted as RTP algorithm). Lastly, the remaining rules, that are not possible to enumerate with higher nodes, are ranked by their effectiveness via a learning-to-rank model called LambdaMART [23], [24]. During the enumeration process, for rules involving higher node counts, the ranking model can be applied to the plan templates first, followed by selecting the top- k templates before proceeding with rule enumeration.

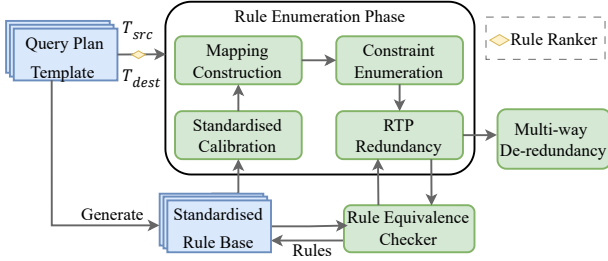


Fig. 3: Architecture of standardized rule enumeration.

Template-Based Rule Enumerator (§IV-A). To address the limitations of brute-force enumeration, SLER employs standardized templates for rule generation, enabling polynomial-time verification complexity for certain redundant templates, unlike WeTune’s exponential complexity. The process begins by selecting source and destination plan templates. Crucially, to reduce enumeration time and handle larger node counts, the LambdaMART ranker (§V) can be optionally applied upfront to select only the top- k template pairs for processing. Following selection, templates are instantiated as queries and validated against *standardized rules*: if a template is deemed rewritable, the generated rules are marked redundant and enumeration for that pair is pruned. For valid templates, the workflow proceeds to constraint enumeration to output specific rewriting rules, which are then passed to the RTP stage for redundancy checking.

RTP-Based Deduplicator (§IV-B). To mitigate verifying complexity, the RTP algorithm is proposed to systematically prune redundancy while preserving the rule base’s optimization capacity. Theoretically, we ensure that all removed rules are redundant. The RTP process works by first verifying the rule, then checking for redundancy among checksum-equivalent rules. If a rule is redundant, RTP halts further enumeration of that rule to reduce subsequent checker calls. Non-redundant rules are included in the rule base, refined by constraint relaxation and a multi-way subsumption algorithm, thereby significantly reducing the final rule set.

LambdaMART Rule Ranker (§V). To identify the most effective rules that maximize system performance, SLER employs the LambdaMART model [23], [24], which is trained to optimize ranking based on Normalized Discounted Cumulative Gain (NDCG). Crucially, the ranker serves a dual purpose to overcome the infeasibility of brute-force enumeration for large-node templates ($n > 7$): it can either (1) pre-filter top- k plan template pairs during the enumeration phase to significantly reduce computational overhead and enable the discovery of rules in higher-node templates; or (2) be applied post-enumeration to predict rule scores (based on features like tree distance and expression complexity) and rank the final complete rule set. This dual capability ensures both scalability and the efficient identification of the most effective rules across all template sizes.

IV. STANDARDIZATION RULES

This section outlines the procedure for query rewrite rule enumeration using standardized rules, with the overall archi-

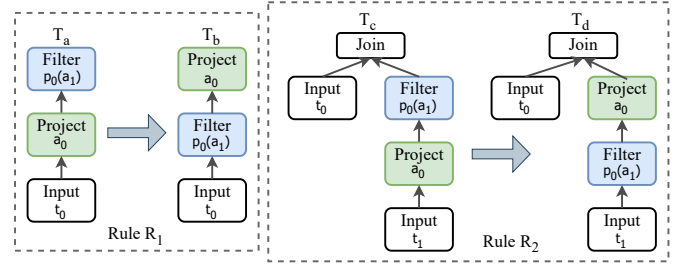


Fig. 4: A case for redundancy rules.

tectural workflow illustrated in Figure 3.

SLER first introduces standardized rule enumeration, which places projection/selection operators below join operators to generate a standardized rule base and reduce verification complexity from exponential to tractable for specific operations. An optional template filtering step via the rule ranker selects top- k pairs for enumeration, cutting computational complexity and enabling exploration of complex templates ($n \geq 7$ nodes). Finally, Reduced by Template Pair (RTP) optimization mitigates redundancy elimination overhead, forming a synergistic workflow for efficient rule generation and refinement.

A. Rule Enumerator

During rule enumeration, structurally similar templates frequently generate excessive redundant rules. For example, in Figure 4, templates T_a and T_b differ only in the vertical positioning of projection and filter operators, while T_c and T_d are nearly identical except for the swapping of two operators. These trivial structural variations introduce substantial redundancy into the rule set. To address this, we propose a **standardized rule** approach to filter out redundant rules at an early stage, minimizing the verification complexity.

Definition 3. A standardized rule $R_n(T_{src}, T_{dest}, M_n, \emptyset)$ is a query rewrite rule (as defined in Definition 1 with an empty constraint set $\emptyset$ and a relationship between T_{src} and T_{dest} restricted to two scenarios: (1). T_{src} and T_{dest} become identical after swapping the positions of two adjacent operators. (2). T_{src} and T_{dest} become identical after removing one operator.

Standardized rules uniform query plans into a canonical structure by swapping or removing operators. A key property of these rules is that any non-standardized rule $R(T_{src}, T_{dest}, M, C)$ is redundant if a standardized rule $R_n(T_{src}, T'_{dest}, M, \emptyset)$ exists.

This property enables efficient filtering of redundant rules, which we informally validate through two cases:

- For a non-standardized rule $R(T_{src}, T'_{dest}, M, C)$ where $T_{dest} = T'_{dest}$, the standardized rule R_n (with an empty constraint set) is more relaxed, making R redundant.
- For a non-standardized rule where $T_{dest} \neq T'_{dest}$, any query optimizable by R can also be optimized by R_n plus another rule R' , such that R becomes redundant in the set $\{R_n, R, R'\}$, i.e., $\forall q \cdot (\text{Rewrite}(\{R_n, R, R'\}, q) = \text{Rewrite}(\{R_n, R'\}, q))$, where $\text{Rewrite}()$ takes a query plan

q and rule set $\{R\}$ to produce an optimized query plan q' such that the output results of q and q' are identical.

Case Study. When employing standardized rules, using Figure 4 as an example, two types of redundant rules can be identified: (1) Both Rule R_1 and Rule R_2 in Fig. 4 involve similar transformations. Here, R_1 acts as the standardized rule R_n because it contains all necessary conditions to transform the query plan represented by template T_c into T_d directly. This renders the non-standardized Rule R_2 redundant, as its effect can be covered by R_1 through rule composition. (2) Since R_1 is established as a standardized rule R_n , any non-standardized rule $R(T_a, T_e, M, C)$ derived from template T_a is identified as redundant. This is because T_a can be first structurally transformed into the canonical form T_b via R_1 , and subsequent optimizations can then be applied to T_b more efficiently. The first type of redundancy can be addressed through de-redundancy after generating the standardized rule base (later in §IV-B). The second type is handled during the rule enumeration.

Reducing Verification Overhead. Unlike WeTune, our approach reduces verification of redundant rules from exponential to polynomial scale, significantly cutting rule checker verification and lowering enumeration overhead.

Traditional rule enumeration generates all possible template pairs (T_{src}, T_{dst}) and verifies their equivalence, which results in exponential complexity $O(((k^n)^2 \times 2^m)^2)$ due to the combinatorial explosion of operators, nodes, and constraints (where n is the number of nodes). In contrast, our standardized rules and subquery push-up constrain template structures, reducing the verification complexity from $O(((k^n)^2)$ to $O(1)$ for targeted, highly-redundant cases. This significant complexity reduction is achieved by eliminating constraint enumeration and limiting the number of verifiable template pairs. The formal logic and complexity analysis for this reduction are provided in the extended version [59]. While this optimization does not apply to all templates, redundant ones are widespread in benchmark and real-world scenarios. Thus, targeted optimizations for these cases yield significant reductions in enumeration overhead (validated in §VI-D).

1) *Generation of Standardized Rules:* This section outlines the methodology for generating standardized rules. Drawing from Definition 3, rules are categorized into two types: Operator Removal (simplifying templates by eliminating redundant operators) and Operator Swapping (adjusting operator positions to align with a canonical structure). Swapping necessitates a predefined priority hierarchy (Project > Distinct > Filter) to avoid non-convergent transformations. This hierarchy reflects analyzed performance impacts: Project is pushed upward to maximize applicability, and Filter is lower-prioritized. A standardized rule swaps positions for validation when a parent node has lower priority than its child. Multi-input operators are excluded because they complicate post-removal transformation structures and enforcing priorities would restrict critical optimization paths (which traditional optimizers already lack rules for). Conversely, Distinct operators are stan-

Algorithm 1: Generating Standardized Rule

Input: Template Library S
Output: Candidate Rule Set NC

```

1  $NC \leftarrow \emptyset$ ;
2 foreach template  $T_{src} \in S$  do
3    $T_{dest} \leftarrow \text{clone}(T_{src})$ ;
4    $M \leftarrow \text{constructNodeMapping}(T_{src}, T_{dest})$ ;
5   foreach node  $n \in T_{dest}$  do
6     if  $n$  is a single-input node then
7       Remove  $n$  from  $T_{dest}$ ;
8       Update  $M$  by removing attributes of  $n$ ;
9        $R_{nc} \leftarrow (T_{src}, T_{dest}, M, \emptyset)$ ;
10       $NC \leftarrow NC \cup \{R_{nc}\}$ ;
11      Restore  $T_{dest}$  and  $M$ ;
12   else
13     Skip to next node;
14    $n_p \leftarrow \text{parent}(n)$ ;
15   if  $n_p \neq \text{null}$  and  $\text{priority}(n_p) < \text{priority}(n)$  then
16     Swap  $n$  and  $n_p$  in  $T_{dest}$ ;
17      $R_{nc} \leftarrow (T_{src}, T_{dest}, M, \emptyset)$ ;
18      $NC \leftarrow NC \cup \{R_{nc}\}$ ;
19     Restore  $T_{dest}$ ;

```

dardized because subquery-based Distinct is often ineffective (focusing on value existence, not result counts), making their standardization valuable for reducing inefficiencies.

Algorithm 1 outlines candidate standardized rule generation: for each template T_{src} in the template library S , a copy T_{dest} is created, and a node mapping M is established between T_{src} and T_{dest} . The algorithm iterates over all nodes in T_{dest} to generate rules for operator removal and swapping. (1) Removal: For single-input node n , remove n from T_{dest} , and the mapping M is updated by excluding n -associated attributes. A new rule $R_{nc} = (T_{src}, T_{dest}, M, \emptyset)$ is added to the candidate set NC , then T_{dest} and M are restored. (2) Swapping: For node n with parent n_p , if n_p has lower priority, swap n and n_p in T_{dest} . A new rule is generated and added to NC , followed by the restoration of T_{dest} .

After generating candidate rules, an equivalence checker validates their correctness, producing a set of valid rules. A deduplication module (details in §IV-B) then removes redundant rules from this set, further enhancing verification efficiency.

2) *Use of Standardized Rule Base:* The standardized rule base is integrated into the framework to optimize query plan templates. Given a source template T_{src} and a destination template T_{dest} , T_{src} is instantiated as P_{src} . The optimizer then applies rules from the standardized rule base S_n to P_{src} . If the resulting optimized plan set P_{dest} is non-empty, T_{src} is deemed redundant (fully optimizable by S_n), and no new rules are generated. If P_{dest} is empty, T_{src} is irreducible via S_n , and the framework proceeds to enumerate minimal equivalent constraints for the pair (T_{src}, T_{dest}) , generating non-redundant rules. This approach efficiently identifies redundant templates and streamlines rule generation.

To guarantee the semantic correctness of all discovered rewrite rules, we employ a formal verification framework leveraging the Z3 SMT solver [19]. Following WeTune [14],

our verifier translates query plan templates and their associated constraints into First-Order Logic formulas. The correctness of a rule is verified using proof by contradiction: the verifier attempts to find a counterexample that satisfies the constraints but violates the equivalence. If the negation of the equivalence assertion is proven to be unsatisfiable (UNSAT) by Z3, the rule is formally confirmed as semantically equivalent.

B. Rule Deduplicator

A key observation is that much redundancy stems from complex mapping constructions and constraint enumerations, which are difficult to filter incrementally during rule generation. Traditional deduplication, relying on nested loops over rules with $O(n^2)$ complexity [14], becomes prohibitively inefficient as template node counts grow: for 4-node templates, de-redundancy exceeds 20 days (Table V), outpacing enumeration itself due to expanded template complexity and search space. To address this, we propose two optimizations: (1) partitioning the rule set into smaller subsets, performing de-redundancy on each subset independently, and merging the results for final subsumption checks; (2) introducing the **Reduce by Template Pair (RTP)** algorithm to enable de-redundancy during the enumeration phase, leveraging template pair properties.

To validate the effectiveness of incremental de-redundancy via subset merging, we briefly prove the following assertion: Given two query rewrite rule sets S and S' where $S' \subseteq S$, if a rule $R \in S'$ is redundant within S' , then R is also redundant within S . The corresponding proof sketch is provided in the extended version [59]

This property enables efficient redundancy elimination through partitioning. Rules are divided into subsets, processed independently, and merged iteratively to yield a de-redundant set equivalent to processing the entire set at once, accelerating the workflow.

To boost enumeration efficiency, we introduce the Reduce by Template Pair (RTP) algorithm, which filters redundancy early at the template pair level. For a rule $R(T_{src}, T_{dest}, M, C)$, its template pair is $P(T_{src}, T_{dest})$; after selecting P , mapping construction and constraint enumeration proceed with embedded deduplication.

The RTP workflow initializes a dedicated rule base S_p for each template pair P during enumeration. For each candidate rule R belonging to P : If R is non-equivalent, it is marked as NEQ and retained. If R is equivalent, its source template is instantiated into a query plan P_{src} , which is optimized using S_p and $S_p \cup \{R\}$, respectively. If the optimizations are identical, R is redundant (marked as Repeat) and discarded; otherwise, R is non-redundant (marked as EQ) and added to S_p .

V. RULE RANKER

This section elaborates on the application of the learning ranking model in the effectiveness ranking of rules and templates. Its architectural workflow is shown in Figure 5. We adopt a LambdaMART model to leverage its capability in handling large-scale datasets and optimizing ranking performance. It covers model design, training procedures, and applications to rule enumeration.

TABLE IV: The formal definitions of operators.

Operator	Expression
Input _r	$f(t) := r(t)$
Proj _a	$f(t) := \sum_{t_l} (f_l(t_l) \times \llbracket t = a(t_l) \rrbracket)$
Sel _{p,a}	$f(t) := f_l(t) \times \llbracket p(a(t)) \rrbracket$
InSubSel _a	$f(t) := f_l(t) \times \llbracket f_r(a(t)) \rrbracket \times \text{not}(\llbracket \text{IsNull}(a(t)) \rrbracket)$
IJoin _{a_l, a_r}	$f(t) := \sum_{t_l, t_r} (\llbracket t = t_l \cdot t_r \rrbracket \times f_l(t_l) \times f_r(t_r) \times \llbracket a_l(t_l) = a_r(t_r) \rrbracket \times \text{not}(\llbracket \text{IsNull}(a_l(t_l)) \rrbracket))$
LJoin _{a_l, a_r}	$f(t) := (\text{IJoin Expr}) + \sum_{t_l, t_r} (\llbracket t = t_l \cdot t_r \rrbracket \times f_l(t_l) \times \llbracket \text{IsNull}(t_r) \rrbracket \times \text{not}(\sum_{t'_r} (f_r(t'_r) \times \llbracket a_l(t_l) = a_r(t'_r) \rrbracket \times \text{not}(\llbracket \text{IsNull}(a_l(t_l)) \rrbracket))))$
RJoin _{a_l, a_r}	(symmetric to LJoin)
Dedup	$f(t) := \llbracket f_l(t) \rrbracket$

A. Model Design

The design of the LambdaMART model for rule effectiveness ranking consists of Template Encoding and Ranking layers, as depicted in Figure 5. This part explains the methods (i.e., encoding process, ranking model, and loss function) employed in these two layers.

1) *Encoding Process*: The ranking model's embedding layer maps each rule $R = (T_{src}, T_{dest}, M, C)$ into a 9-dimensional (9-dim) space to capture structural differences between query plans. This vectorization process extracts critical structural information from each template, ensuring both efficacy and efficiency in identifying plan transformations.

SLER builds upon tree convolutional models [39], [41], [60], [61] but enhances the representation by omitting estimated costs to focus solely on node properties. As shown in Figure 5, it converts plan trees into vectorized representations where nodes are one-hot encoded by operator type, using convolutions to capture hierarchical relationships.

Unlike prior works [39], [41], [61], our model excludes external factors like cardinalities or table statistics, concentrating on intrinsic operator properties. Convolutional kernels process each node and its children, followed by dynamic pooling to generate the final 9-dim embedding. These vector representations for T_{src} and T_{dest} are then combined into a feature vector for ranking.

The encoding phase converts raw query rewrite rules into a feature-rich representation amenable to machine learning. This stage produces a feature vector encompassing nine attributes:

L2 Distance: Measures the ① structural transformation magnitude between T_{src} and T_{dest} , computed as: $\text{L2 Distance} = \sqrt{\sum_{i=1}^n (e_{src,i} - e_{dest,i})^2}$, where $e_{src,i}$ and $e_{dest,i}$ are components of tree embedding vectors.

Cosine Similarity: Assesses the ② angular similarity between template embeddings: $\text{Cosine Similarity} = \frac{\sum_{i=1}^n e_{src,i} \cdot e_{dest,i}}{\sqrt{\sum_{i=1}^n e_{src,i}^2} \cdot \sqrt{\sum_{i=1}^n e_{dest,i}^2}}$.

Expression Complexity Features: These include metrics derived from operator expressions in Table IV, averaged over primary operators, capturing syntactic and semantic intricacy: ③ `expr_length` as the string length of the expression, indi-

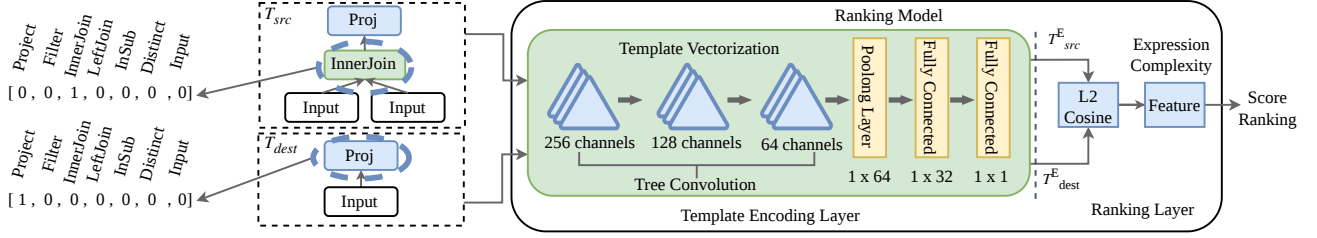


Fig. 5: Architecture of rule effectiveness ranking.

cating syntactic volume; ④ `sum_count` as the count of $\sum$ operators, quantifying aggregation depth; ⑤ `times_count` as the count of $\times$ operators, quantifying multiplicative dependencies; ⑥ `neg_count` as the count of $\neg$ operators, quantifying logical negation; ⑦ `bracket_count` as the number of `[ ]` bracket pairs, quantifying conditional branching; ⑧ `var_count` as the number of variables (e.g., `t`, `r`, `f`, `a`), refined as $\max(\text{raw count}, \text{null count})$ to handle abstract templates; and ⑨ `complexity` as the total operator count (`sum` + `times` + `neg` + `bracket`), serving as a proxy for verification overhead.

Tree embeddings are derived using a neural network that processes the abstract syntax tree (AST) of each template, capturing hierarchical relationships.

2) *Ranking Model*: The ranking model employs Gradient Boosted Decision Trees (GBDT) [62] to enhance predictive accuracy through an iterative ensemble of decision trees. Each successive tree is trained via gradient descent to minimize residuals, resulting in a final prediction formed by the weighted sum of all trees while preventing overfitting through a controlled learning rate.

We implement GBDT using the LightGBM library, specifically utilizing the LambdaMART algorithm for scoring and ranking tasks. The model processes the 9-dim TCNN embeddings of T_{src} and T_{dest} (§V-A1), which are integrated with relevance labels to construct a training dataset optimized for identifying effective rewrite rules.

3) *Loss Function*: The ranking model maximizes the likelihood of outputting the correct order within the rule set to select the best top plans; thus, its loss function is designed to achieve this goal. It uses the `lambdarank` objective in LightGBM, which optimizes Normalized Discounted Cumulative Gain at n ($\text{NDCG}@n$), a metric evaluating the quality of the top- n ranked rules.

$\text{NDCG}@n$ focuses on the top n positions, with the calculation formula as follows: $\text{NDCG}@n = \frac{\text{DCG}@n}{\text{IDCG}@n}$ where $\text{DCG}@n = \sum_{i=1}^n \frac{2^{\text{rel}_i - 1}}{\log_2(i+1)}$ is the Discounted Cumulative Gain, rel_i is the relevance score (1 and 0 indicating a effective and ineffective rules in reducing execution time), and $\log_2(i+1)$ is a position-based discount factor. $\text{IDCG}@n$ is the maximum possible DCG for the top n ranks, computed using the optimally ordered relevance scores.

LambdaRank: The loss is determined by the Lambda weight (λ_{ij}), which reflects the change in NDCG when the ranks of

two items are swapped.

Lambda weight (λ_{ij}): A weighting factor that measures the impact of swapping the ranks of two rules R_i and R_j on the $\text{NDCG}@n$ metric, defined as: $\lambda_{ij} = \frac{\partial \text{NDCG}@n}{\partial s_i} - \frac{\partial \text{NDCG}@n}{\partial s_j}$ where $\frac{\partial \text{NDCG}@n}{\partial s_k}$ is the gradient of $\text{NDCG}@n$ with respect to the score s_k of rule R_k , approximated based on the relevance difference and position discount. Rule pairs are adjusted in their relative order to ensure that rules with higher relevance scores are ranked above those with lower scores.

Relevance score (rel_i): A binary label indicating a rule’s effectiveness, derived from execution time reduction, where 1 denotes a beneficial rule and 0 denotes a non-beneficial one.

LambdaMART employs a pairwise ranking approach to minimize discrepancies between predicted scores and ground-truth labels. For any pair (R_i, R_j) , the model adjusts scores to satisfy $s_i > s_j$ whenever $\text{rel}_i > \text{rel}_j$. This sigmoid-based loss is weighted by λ_{ij} and optimized via gradient descent within the GBDT framework.

B. Model Training

Unlike conventional approaches that depend on pre-trained cost models or synthetic workloads, our training uses real-world SQL transaction data, focusing solely on intrinsic rewrite rule differences (excluding external factors like table data, cardinality, or estimated costs). This part outlines the pipeline for real transaction workloads: templated SQL statements (matching rule source templates) are rewritten using applicable rules, with effectiveness validated via runtime comparisons in a production database. Rules reducing latency are labeled 1, ineffective ones 0, ensuring the model learns from practical outcomes rather than synthetic estimates.

Features are extracted exclusively from rule representations (§V-A1) and integrated into a LightGBM Dataset with one group $[\text{len}(X_{\text{train}})]$, mirroring a unified query context to support the `lambdarank` objective for pairwise rule optimization using relevance labels.

Training employs the LambdaMART algorithm within LightGBM, configured to rank the top k beneficial rules effectively. The process iterates over 100 boosting rounds, constructing an ensemble of decision trees to adjust predicted scores $s_k = \sum_{t=1}^{100} f_t(F_k)$, where F_k is the feature vector of rule R_k and f_t is the t -th tree’s prediction. The `lambdarank` loss function maximizes Normalized Discounted Cumulative

Gain (NDCG@ k), as defined in the mathematical framework of §V-A2, with the Lambda weight λ_{ij} , detailed in the same section, guiding gradient descent to enforce correct ordering. Upon completion, the model ranks rules adaptively for specific workloads, enhancing efficiency without reliance on external cost models.

C. Applications To Rule Enumeration

To accelerate rule discovery for large-node templates, the ranker acts as an optional pre-filter. It scores potential source-destination pairs based on their structural features and prioritizes only the top- k candidates for subsequent validation and enumeration. This targeted strategy significantly reduces the computational burden of exhaustive search while focusing on the most promising transformations.

Top- k filtering enables configurable trade-offs between computational efficiency and high-value rule capture. While larger values of k extend enumeration time, they increase the coverage of complex, multi-node optimizations. Conversely, a smaller k enables the exploration of large-node templates (e.g., $n \geq 7$) that are otherwise computationally infeasible. To avoid missing critical optimizations, our ranker employs a multi-objective approach by prioritizing templates based on their structural generality and simplification potential. This strategy ensures that the most impactful rules are concentrated in the top-ranked results, making the optimization quality robust to the specific choice of k . This lets practitioners tune k for search depth-enumeration overhead balance, preserving the most general and effective rules even with restricted k .

Furthermore, our structure-aware LTR model is workload-independent, allowing the pre-trained ranker to generalize across diverse schemas without re-training.

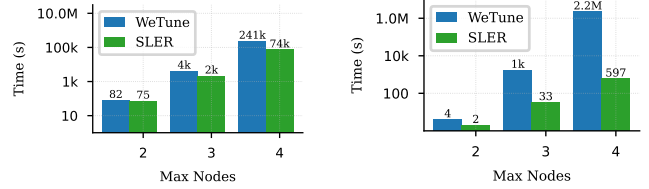
VI. EVALUATION

The evaluation aims to answer the following questions:

Q1: Compared to SOTA WeTune, what is the efficiency of SLER in enumerating effective rules? Specifically, does it demonstrate fast performance on small-scale tasks, and is it feasible to apply it to large-scale scenarios? (§VI-B) **Q2:** Compared to SOTA WeTune, what is the effectiveness of SLER in enumerating rules? This includes aspects such as whether it can enumerate a greater number of rules and effective rules that were not reported previously. (§VI-C) **Q3:** What are the individual and combined contributions of SLER’s proposed components to its overall efficiency and effectiveness? This details the ablation study of our proposed rule enumerator, deduplicator, and ranker. (§VI-D) **Q4:** How do the rules discovered by SLER translate into actual query latency reduction in real-world database systems, and how effectively does the ranker prioritize high-impact rules? (§VI-E)

A. Experimental Setup

Testbed. Implemented on WeTune codebase, SLER is publicly available at [59] for reproduction purposes. The experiments were conducted on a server equipped with an Intel(R) Xeon(R) Silver 4216 processor (64x 2.10 GHz), featuring 16



(a) Enumeration time

(b) De-redundancy time

Fig. 6: Efficiency evaluation of SLER (log scale).

physical cores and 32 threads, with 376 GB of memory. The thread count was set to 16 to minimize context-switching overhead, thereby optimizing performance for both rule generation and model training. SQL Server 2019 [63] is deployed for model training of SLER’s ranker.

Baseline. To address Q1 and Q2, SLER is primarily compared against SOTA WeTune’s [14]. The comparison focuses on efficiency, with key metrics including enumeration time, verification calls, the maximum number of nodes in templates, and the improvement in identifying useful rules. For Q3, we conducted an ablation study on our proposed rule enumerator (§IV-A), deduplicator (§IV-B), and ranker (§V). For Q4, we conducted an end-to-end evaluation on our rewritten queries.

Generating Rules. SLER employs a template-optimized enumeration algorithm to generate query plan templates with 5 and 6 nodes, resulting in 26,508 and 652,808 new non-redundant and high-potential rules, respectively. In general, WeTune is infeasible to enumerate rules for templates with more than 4 nodes.

Evaluating Rules. The optimization performance of the LambdaMART model is assessed by rewriting real-world SQL queries using rule libraries both before and after ranking, followed by quantifying the ranking positions of rules employed during actual SQL execution. The model is trained on rules validated within a SQL Server environment, where rules that reduce execution time are assigned a relevance score of 1. We compare the model’s performance against a baseline of random rule ordering, employing the NDCG@ k metric (optimized during training via the LambdaRank objective function) to quantify the accuracy of ranking effective rules in top positions. This evaluation approach ensures that the model prioritizes rules with discussed optimization potential, which aligns with the goal of enhancing query performance.

Datasets/Queries Statistics. We evaluate SLER on a large-scale workload consistent with existing benchmarks [14], comprising 11,518 unique SQL queries. This includes 8,518 queries directly sourced from WeTune [14], which were collected from 15 open-source applications (e.g., *Discourse*, *Redmine*). Additionally, we incorporate 3,000 commercial queries from YashanDB [64], featuring intricate relational patterns for financial systems. Commercial queries are kept private due to proprietary restrictions. For the end-to-end performance evaluation, we deploy these queries on SQL Server populated

TABLE V: Efficiency evaluation in enumeration and de-redundancy. A “timeout” is an estimated runtime exceeding 10 years, signifying scenarios where rule enumeration or verification becomes computationally infeasible.

Max nodes in the template		2	3	4	5	6
Enumeration Duration	WeTune	82s	1h 11m	67h 4m	timeout	timeout
	SLER	72s	39m 22s	22h 59m	timeout	timeout
	(w.o. RTP)	75s	36m 46s	20h 49m	29d 23h	395d
De-redundancy Duration	WeTune	4s	28m 53s	20d 19h	timeout	timeout
	SLER	3s	2m 27s	33m 33s	timeout	timeout
	(w.o. RTP)	2s	33s	9m 57s	10h 24m	62h 24m

with 10M+ rows for the Discourse dataset.

B. Efficiency

This part compares SLER to SOTA WeTune in terms of enumeration and de-redundancy time. SLER beats WeTune in both small-scale and large-scale rule enumeration, with faster execution for small-node rules and feasible scalability for large-node scenarios.

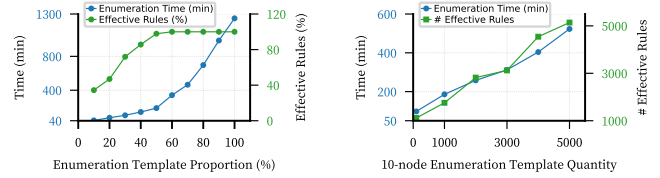
Small-Scale Efficiency. For rules with 2–4 nodes, SLER reduces enumeration time significantly. As shown in Figure 6, SLER completes 2-, 3-, and 4-node enumeration in 72 seconds (87.8% of WeTune’s time), 2,362 seconds (55.14% of WeTune’s time), and 82,763 seconds (34.27% of WeTune’s time), respectively. Also, SLER completes 2-, 3-, and 4-node de-redundancy in 2 seconds (50% of WeTune’s time), 33 seconds (2% of WeTune’s time), and 597 seconds (<1% of WeTune’s time), respectively.

Large-Scale Feasibility. WeTune fails to enumerate rules involving $n \geq 5$ (timeout), and even SLER takes 395 days for $n = 6$ (Table V). To enable scalable enumeration for $n \geq 7$ templates, the framework leverages its effectiveness-driven ranker design. This strategy uses SQL template features and LambdaMART training to pre-sort rules, prioritizing the top n rules for validation (where n is dynamically determined). This targeted prioritization significantly reduces validation overhead, making scalable (but incomplete) enumeration for $n \geq 7$ nodes feasible. Figure 7a highlights an optimal efficiency trade-off: a 40% template filtering rate yields over 80% effective rule coverage in just **143 minutes**. This configuration offers high efficiency under a controlled trade-off, making it a practical choice for scalability. Figure 7b further demonstrates SLER’s scalability to complex 10-node rules. When scaling up to 5k templates, the system discovers over 5.1k effective rules in 523 minutes, confirming its strong scalability even for significantly larger rule templates.

SLER’s small-scale efficiency and large-scale feasibility derive from standardized templates and RTP, which reduce redundancy and verification overhead, critical as node counts and redundant rules grow.

C. Effectiveness

This part compares SLER to SOTA WeTune in terms of the number and effectiveness of rules. SLER demonstrates



(a) 4-node rules

(b) 10-node rules

Fig. 7: Rule Enumeration Efficiency and Effectiveness Across Different Template Proportions and Node Counts.

superior effectiveness by enumerating more rules, including large-node rules, and effective rules that were not reported previously.

Rule Quantity and Scalability. WeTune is limited to 4-node rules, while SLER expands the rule base by generating 26,508 new 5-node rules and 652,808 new 6-node rules, reaching 679,316 total rules, 98.25x larger than WeTune’s set. For rules involving $n \geq 7$ nodes, where full enumeration is computationally infeasible, SLER employs an effectiveness-driven template filtering strategy using the LambdaMART ranker. As demonstrated in Figure 7b, we can sacrifice completeness to node-size scalability. A larger set of rules increases the proportion of effective rules but extends enumeration time, offering a tunable trade-off between efficiency and rule coverage.

Table VII presents a representative subset of 8 standardized rules, encompassing nearly all rule types. Rules 1 and 2 belong to the operator exchange category, while rules 3 and 4 are operator deletion rules. Rules 5–6 focus on join operations, and rules 7–8 target subquery transformations. This set is representative due to its coverage of diverse query rewriting scenarios. As detailed in §IV-A, these standardized rules eliminate redundancies by leveraging structural similarity while preserving semantically distinct variants.

Rule Quality. SLER’s rules enable direct, one-step optimization of queries, where WeTune requires multiple steps. For example in Table I, Query q_4 is optimized to q_6 using a single 5-node rule from SLER (Rule 2 from Table VI), whereas WeTune needs four steps of rule applications. For query q_7 , WeTune can only produce a non-minimal form. This stems from its primary design for query patterns with four or fewer nodes. Also, Table VI shows more new rules, none of which are discovered by the WeTune.

D. Ablation Study

This part evaluates SLER’s three core components’ contribution, i.e., the enumerator, deduplicator, and ranker, and their synergistic gains when combined.

Enumerator. As shown in Table VIII, Standardized templates (OP1) reduce verification complexity from exponential to polynomial for redundant templates, cutting 3- and 4-node enumeration time by 44.86% and 65.73%, respectively, compared to no optimization (No OPT). This component alone reduces verification calls by 50.24% and 68.01% for 3- and

TABLE VI: Higher ranking effective rules identified by SLER, none of which were discovered by WeTune. Each t_i signifies an input table, while each a_i denotes a list of attributes, and each p_i indicates a predicate. Multiple instances of the same symbol (i.e., t_i , a_i , p_i) illustrate equivalent constraints. Each $(t_i.a_j)$ denotes a constraint SubAttrs(a_j , a_{t_i}). Additional types of constraints are detailed in the Extra Constraints column. More representative rules are available in [59]

No.	Source Plan Template	Destination Plan Template	Extra Constraints
1	InSub $_{a_4}(t_0, \text{Proj}_{a_1,s_1}(\text{Sel}_{p_0,a_2}(\text{InSub}_{a_1}(t_1, \text{Proj}_{a_0,s_0}(t_2))))$	InSub $_{a_4}(t_0, \text{Proj}_{a_0,s_0}(\text{Sel}_{p_0,a_2}(\text{LJoin}_{a_1,a_0}(t_1, t_2))))$	
2	InSub $_{a_2}(\text{LJoin}_{a_0,a_0}(t_0, \text{Proj}_{a_0,s_0}(\text{InSub}_{a_0}(t_0, t_0)), \text{Proj}_{a_2,s_1}(t_0)), t_0)$	IJoin $_{a_0,a_0}(\text{Proj}_{a_0,s_0}(t_0), \text{Proj}_{a_2,s_1}(t_0))$	NotNull(t_0, a_2)
3	InSub $_{a_1}(\text{LJoin}_{a_1,a_0}(t_0, \text{Proj}_{a_1,s_0}(\text{InSub}_{a_0}(t_0, t_0))), \text{Proj}_{a_0,s_1}(t_0))$	IJoin $_{a_1,a_0}(t_0, \text{Proj}_{a_1,s_0}(t_0))$	NotNull(t_0, a_0)
4	LJoin $_{a_0,a_0}(\text{Proj}_{a_1,s_0}(\text{Sel}_{p_0,a_0}(t_0)), \text{Proj}_{a_0,s_1}(\text{Sel}_{p_0,a_0}(t_0)))$	IJoin $_{a_0,a_0}(\text{Proj}_{a_1,s_0}(t_0), \text{Proj}_{a_0,s_1}(t_0))$	NotNull(t_0, a_0)
5	InSub $_{a_0}(\text{LJoin}_{a_0,a_1}(\text{Proj}_{a_1,s_0}(\text{InSub}_{a_0}(t_0, t_0)), \text{Proj}_{a_0,s_1}(t_0)), t_0)$	IJoin $_{a_0,a_1}(\text{Proj}_{a_1,s_0}(t_0), \text{Proj}_{a_0,s_1}(t_0))$	NotNull(t_0, a_0)
6	IJoin $_{a_5,a_4}(\text{Proj}_{a_1,s_1}(\text{LJoin}_{a_1,a_1}(\text{Proj}_{a_0,s_0}(t_0, t_0)), \text{Proj}_{a_4,s_2}(t_2)))$	IJoin $_{a_5,a_0}(\text{Proj}_{a_1,s_1}(\text{LJoin}_{a_1,a_1}(\text{Proj}_{a_0,s_0}(t_0, t_0)), \text{Proj}_{a_4,s_2}(t_2)))$	
7	Sel $_{p_0,t_0,a_1}(\text{IJoin}_{a_0,a_1,t_1,a_1}(t_0, \text{Proj}_{t_1,a_1}(\text{LJoin}_{a_1,a_1,t_2,a_0}(t_1, \text{Dedup}(\text{Proj}_{t_2,a_0}(t_2))))$	IJoin $_{a_0,a_1,t_1,a_1}(t_0, \text{Proj}_{t_1,a_1}(\text{Sel}_{p_0,t_1,a_1}(\text{InSub}_{t_1,a_1}(t_1, \text{Proj}_{t_2,a_0}(t_2))))$	
8	Sel $_{p_0,a_0}(\text{InSub}_{a_0}(\text{LJoin}_{a_0,a_0}(\text{Proj}_{a_0,s_0}(\text{Sel}_{p_0,a_0}(t_0)), t_0), t_0))$	LJoin $_{a_0,a_0}(\text{Proj}_{a_0,s_0}(\text{Sel}_{p_0,a_0}(t_0)), t_0)$	NotNull(t_0, a_0); Unique(t_0, a_0)
9	Proj $_{a_5,s_1}(\text{InSub}_{a_0}(\text{LJoin}_{a_0,a_1}(t_0, t_1), \text{Proj}_{a_2,s_0}(\text{Sel}_{p_0,a_2}(t_2))))$	Proj $_{a_5,s_1}(\text{InSub}_{a_0}(t_0, \text{Proj}_{a_2,s_0}(\text{Sel}_{p_0,a_1}(\text{LJoin}_{a_1,a_2}(t_1, t_2))))$	Unique(t_0, a_5)
10	IJoin $_{a_0,a_1}(\text{Proj}_{a_1,s_0}(\text{InSub}_{a_0}(t_0, t_0)), \text{Proj}_{a_0,s_1}(\text{Sel}_{p_0,a_1}(t_0)))$	IJoin $_{a_0,a_1}(\text{Proj}_{a_0,s_0}(\text{Sel}_{p_0,a_0}(t_0)), \text{Proj}_{a_1,s_1}(\text{Sel}_{p_0,a_0}(t_0)))$	NotNull(t_0, a_0)
11	InSubFilter $_{a_5}(\text{InnerJoin}_{a_1,a_2}(\text{Input}_{t_0}, \text{Proj}_{a_0,s_0}(\text{Input}_{t_1})), \text{Proj}_{a_5,s_1}(\text{LeftJoin}_{a_1,a_2}(\text{Input}_{t_0}, \text{Input}_{t_1})))$	LeftJoin $_{a_1,a_2}(\text{Input}_{t_0}, \text{Proj}_{a_0,s_0}(\text{Input}_{t_1}))$	NotNull(t_0, a_1); NotNull(t_0, a_5); Reference(t_0, a_1, t_1, a_2)

TABLE VII: Representative examples of standardized rules. Proj, IJoin, LJoin, Proj*, and InSub stand for a predicate, InnerJoin, LeftJoin, Distinct, and SubQuery, respectively.

No.	Source Plan Template	Destination Plan Template
1	Filter $_{p_0,a_1}(\text{Proj}_{a_0}(t_0))$	Proj $_{a_0}(\text{Filter}_{p_0,a_1}(t_0))$
2	Filter $_{p_0,a_1}(\text{Proj}_{a_0}^*(t_0))$	Proj $_{a_0}^*(\text{Filter}_{p_0,a_1}(t_0))$
3	Proj $_{a_1}(\text{Proj}_{a_0}(t_0))$	Proj $_{a_1}(t_0)$
4	Proj $_{a_1}^*(\text{Proj}_{a_0}(t_0))$	Proj $_{a_1}^*(t_0)$
5	Proj $_{a_3}(\text{LJoin}_{a_1,a_2}(\text{Proj}_{a_0}(t_0), t_1))$	Proj $_{a_3}(\text{LJoin}_{a_1,a_2}(t_0, t_1))$
6	Proj $_{a_3}(\text{LJoin}_{a_1,a_2}(\text{Proj}_{a_0}(t_0), t_1))$	Proj $_{a_3}(\text{LJoin}_{a_1,a_2}(t_0, t_1))$
7	InSub $_{a_1,a_2}(\text{Proj}_{a_0}(t_0), t_1)$	Proj $_{a_0}(\text{InSub}_{a_1,a_2}(t_0, t_1))$
8	InSub $_{a_1,a_2}(\text{Proj}_{a_0}^*(t_0), t_1)$	Proj $_{a_0}^*(\text{InSub}_{a_1,a_2}(t_0, t_1))$

TABLE VIII: Evaluation of SLER’s Enumerator (OPT1) and Deduplicator (OPT2) in terms of the number of equivalence verification calls and enumeration time.

Max Nodes	#Verification Calls			Enumeration Time (s)		
	No OPT	OPT1	OPT1+OPT2	No OPT	OPT1	OPT1+OPT2
2	24,015	48.23%	45.98%	82	87.80%	91.46%
3	5,839,699	49.76%	45.74%	4,284	55.14%	51.48%
4	314,860,926	31.99%	28.97%	241,457	34.27%	31.05%

TABLE IX: Standardized rule generation overhead.

Max Nodes in the template	2	3	4	5	6
Enumeration overhead	1.47s	4.69s	33.20s	5m 42s	38m 44s
De-redundancy overhead	2.10s	2.93s	4.45s	2m 44s	16m 28s
Total overhead	3.57s	7.62s	37.65s	8m 26s	55m 12s
Overhead ratio	4.64%	0.34%	0.0498%	0.0193%	0.0104%
Number of standardized rules	5	25	77	204	1030

4-node templates, eliminating trivial structural variations (e.g., operator swaps) in advance.

Deduplicator. As shown in Table VIII, RTP (OPT2) prunes redundant rules during enumeration, further reducing checker calls by 3.82–4.02% for 2–4 node templates and cutting 4-node deduplication time from 33 minutes (SLER without RTP) to <10 minutes (SLER), as shown in Table V. It ensures only redundant rules are removed, preserving optimization capacity while minimizing overhead. Nodes exceeding 5 are excluded, as the baseline fails to complete enumeration for such cases.

Table IX summarizes the overhead of enumeration and

TABLE X: Evaluation of SLER’s ranker in terms of trigger rate and rule usage.

Rule Top	Rule End	Trigger Rate (%)	Top Rule Usage(%)	End Rule Usage(%)
Top 50	End 150	100	52	0
Top 500	End 200	100	34.67	0
Top 1000	End 0	100	11.4	0
Top 0	End 1000	0	0	0
Random 1000		0	0.2	0
Random 90000		0.07	13	0

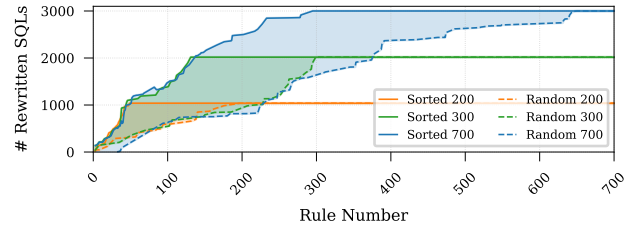


Fig. 8: Evaluation of SLER’s ranker in terms of cumulative rewritten SQLs by adding rules.

deduplication. Generating a standardized rule base for templates with up to 6 nodes incurs minimal overhead due to its linear complexity (traversing each template once, proportional to template count). Even as the maximum node count increases (raising average nodes per template), the enumeration and de-redundancy overhead remain insignificant (<0.05% for 4- or more-node enumeration).

Ranker. The ranker filters trivial rules, ensuring top-ranked rules deliver meaningful performance gains. Validation via the SQL Server optimizer shows that effective optimization rules are concentrated in top-ranked sets. As shown in Table X, combinations of 50–1,000 top-ranked rules (paired with 0–200 low-ranked rules) achieve a 100% trigger rate, with top-rule usage rates (proportion of top-ranked rules applied by the optimizer) of 11.4–52.0% and no low-ranked rules applied. In contrast, random selection proves highly ineffective: a set of 1,000 random rules triggers zero optimizations (0% trigger rate). Even when the pool is expanded to 90,000 random rules, the trigger rate remains negligible at 0.07%. The superiority of our learned ranker is even more evident when examining the efficiency of rule application. Random

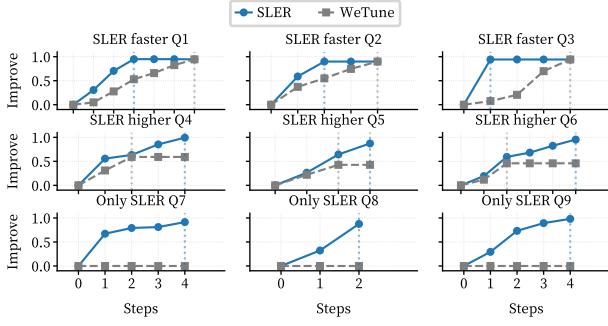


Fig. 9: End-to-end latency improvement comparison.

experiments (average of 5 repeats) reinforce that, as shown in Figure 8, in tests with 700 mixed rules, sorted subsets (700, 300, 200) completed rewrites for 3,000 queries by the 295th, 131st, and 49th rules, respectively, far outperforming random subsets (requiring 650th, 299th, 197th rules). This stems from prioritizing effective rules early. Notable valid previous-not-found rules (Table VI) are consistently top-ranked and frequently applied, significantly boosting rewriting efficiency for complex queries (e.g., Table I). Also, the ranker’s overhead (<1 min) is negligible for overall performance.

E. End-to-End Performance Impact of Rewrite Rules

To further evaluate SLER against no rewrite rules and WeTune, based on our evaluation, 9 representative queries are selected to demonstrate superior results as shown in Figure 9, which are categorized into three scenarios.

Scenario A: Faster Convergence (Q1–Q3). In these cases, both systems reach the same optimal plan. However, SLER achieves this in significantly fewer steps. This is because SLER’s large-node rules act as “composite transformations” that encapsulate multiple small-node rules. While WeTune must apply several atomic rules sequentially, SLER can bridge the gap in a single step, avoiding complex intermediate states.

Scenario B: Higher Optimization (Q4–Q6). SLER achieves a higher improvement ratio than WeTune, as SLER proactively enumerates and verifies complex patterns that exceed the 4-node threshold. WeTune stalls at local optima because its small-rule library cannot perceive the deeper structural simplifications possible with larger templates.

Scenario C: Unique Optimization Capability (Q7–Q9). In these challenging cases, WeTune fails to provide any optimization (0% improvement), whereas SLER identifies significant simplification opportunities. This highlights the superior coverage of our rule library. Many intricate query patterns require specific multi-operator templates (e.g., large-node composite structures such as multi-level join-filter-join redundancies involving 5–9 nodes) that only SLER’s active discovery framework can successfully enumerate and verify. Note that experiments on MySQL yielded similar results and are omitted for brevity.

Performance Improvement for Top Rules. Evaluation results confirm top-ranked rules from SLER drive exceptional

performance gains: the top 500 effective rules achieve a 98.6% average improvement, vastly outperforming the bottom 50% (11.9% average). This stark disparity validates the ranking mechanism’s strong ability to prioritize high-value, latency-reducing rules from the large enumeration space, demonstrating the ranker’s high real-world efficacy in identifying impactful transformations that deliver positive optimization for evaluated workloads.

VII. DISCUSSION

Scope and Limitations. While SLER shows strong performance in scaling rule discovery, its current implementation has specific scopes. (1) *Operator Support*: We currently support core relational operators, including Input, Project, Filter, (Inner/Left/Right) Join, InSub, and Distinct. These form the backbone of most SPJ-style (Select-Project-Join) queries. (2) *Beneficiary Patterns*: Our framework is particularly effective for ORM-generated SQL queries, which often feature deeply nested subqueries and large-scale structural redundancies that exceed the optimization window of traditional 4-node rule sets. (3) *Extensibility*: To extend SLER to aggregates, group-by, and window functions, two main requirements must be met: first, defining the First-Order Logic (FOL) semantics for these operators to enable Z3 verification; second, establishing the corresponding standardized templates for enumeration. This represents a promising direction for future work to broaden SLER’s applicability to complex OLAP workloads.

The Practical Bound of Enumeration. While the enumeration space grows exponentially with the number of nodes, we observe that nodes between 7 and 9 represent a “spot” for rule discovery. Beyond 10 nodes, rules become overly specific and rarely triggered in real-world queries. By focusing on these complex yet reusable patterns, SLER avoids the infinite pursuit of extremely large templates while significantly outperforming the 4-node limit of SOTA works. Most importantly, the enumeration is a one-time process. Once the rules are discovered, they can be directly integrated into DBMSs.

VIII. CONCLUSION

This paper proposes SLER, a rewrite framework via standardized templates, RTP deduplication, and LambdaMART ranking. It overcomes the “small-to-large” hypothesis, and cuts verification overhead by pruning redundancy, while prioritizing effective rules, reducing redundant template enumeration complexity from exponential to polynomial. Experimental results demonstrate that after training on over open-source and commercial queries, SLER outperforms SOTA methods in both efficiency and rule effectiveness.

IX. ACKNOWLEDGEMENT

This work is sponsored by NSFC 62472289 and Guangdong Province Key Laboratory of Popular High Performance Computers 2017B030314073. Yuxing Chen, Rui Mao, and Jianbin Qin are corresponding authors.

AI-GENERATED CONTENT ACKNOWLEDGEMENT

We used Gemini/Yuanbao to edit the main text. AI was only used for polishing the writing. We are responsible for all the materials presented in this work.

REFERENCES

- [1] X. Tang, S. Wu, D. Zhang, F. Li, and G. Chen, "Detecting logic bugs of join optimizations in dbms," *Proceedings of the ACM on Management of Data*, vol. 1, no. 1, pp. 1–26, 2023.
- [2] Q. Bai, S. Alsudais, and C. Li, "Querybooster: Improving sql performance using middleware services for human-centered query rewriting," *Proc. VLDB Endow.*, vol. 16, no. 11, p. 2911–2924, Jul. 2023.
- [3] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, "Access path selection in a relational database management system," in *Proceedings of the 1979 ACM SIGMOD international conference on Management of data*, 1979, pp. 23–34.
- [4] D. Wang, Y. Chen, C. Jiang, A. Pan, W. Jiang, S. Wang, H. Lei, C. Zhu, L. Zheng, W. Lu, Y. Chai, F. Zhang, and X. Du, "TXSQL: lock optimizations towards high contented workloads," in *Companion of the 2025 International Conference on Management of Data, SIGMOD/PODS 2025, Berlin, Germany, June 22–27, 2025*, V. Markl, J. M. Hellerstein, and A. Abouzied, Eds. ACM, 2025, p. 675–688. [Online]. Available: <https://doi.org/10.1145/3722212.3724457>
- [5] G. Graefe, "Rule-based query optimization in extensible database systems," University of Wisconsin-Madison Department of Computer Sciences, Tech. Rep., 1987.
- [6] G. Graefe and W. J. McKenna, "The volcano optimizer generator: Extensibility and efficient search," in *Proceedings of IEEE 9th international conference on data engineering*. IEEE, 1993, pp. 209–218.
- [7] G. Graefe, "The cascades framework for query optimization," *IEEE Data Eng. Bull.*, vol. 18, no. 3, pp. 19–29, 1995.
- [8] G. Graefe and D. J. DeWitt, "The exodus optimizer generator," in *Proceedings of the 1987 ACM SIGMOD international conference on Management of data*, 1987, pp. 160–172.
- [9] A. Y. Levy and I. S. Mumick, "Query optimization by predicate move-around," Aug. 19 1997, uS Patent 5,659,725.
- [10] I. S. Mumick, S. J. Finkelstein, H. Pirahesh, and R. Ramakrishnan, "Magic is relevant," *ACM SIGMOD Record*, vol. 19, no. 2, pp. 247–258, 1990.
- [11] M. Muralikrishna et al., "Improved unnesting algorithms for join aggregate sql queries," in *VLDB*, vol. 92, 1992, pp. 91–102.
- [12] H. Pirahesh, J. M. Hellerstein, and W. Hasan, "Extensible/rule based query rewrite optimization in starburst," *ACM Sigmod Record*, vol. 21, no. 2, pp. 39–48, 1992.
- [13] P. Seshadri, J. M. Hellerstein, H. Pirahesh, T. C. Leung, R. Ramakrishnan, D. Srivastava, P. J. Stuckey, and S. Sudarshan, "Cost-based optimization for magic: Algebra and implementation," in *Proceedings of the 1996 ACM SIGMOD international conference on Management of data*, 1996, pp. 435–446.
- [14] Z. Wang, Z. Zhou, Y. Yang, H. Ding, G. Hu, D. Ding, C. Tang, H. Chen, and J. Li, "Wetune: Automatic discovery and verification of query rewrite rules," in *Proceedings of the 2022 International Conference on Management of Data*, 2022, pp. 94–107.
- [15] TPC, "TPC-H," <http://www.tpc.org/tpch/>, August 2024.
- [16] —, "TPC-DS," <http://www.tpc.org/tpcds/>, August 2024.
- [17] PostgreSQL. [Online]. Available: <https://www.postgresql.org/>
- [18] Z. Li, H. Yuan, H. Wang, G. Cong, and L. Bing, "Llm-r2: A large language model enhanced rule-based rewrite system for boosting query efficiency," *Proceedings of the VLDB Endowment*, vol. 17, no. 11, pp. 3212–3225, 2024.
- [19] L. de Moura and N. Bjørner, *Z3: An Efficient SMT Solver*, Jan 2008, p. 337–340. [Online]. Available: http://dx.doi.org/10.1007/978-3-540-78800-3_24
- [20] P. Pudlák, "The lengths of proofs," in *Studies in Logic and the Foundations of Mathematics*. Elsevier, 1998, vol. 137, pp. 547–637.
- [21] J. Krajíček, *Proof complexity*. Cambridge University Press, 2019, vol. 170.
- [22] S. A. Cook and R. A. Reckhow, "The relative efficiency of propositional proof systems," *The journal of symbolic logic*, vol. 44, no. 1, pp. 36–50, 1979.
- [23] C. J. Burges, "From ranknet to lambdarank to lambdamart: An overview," *Learning*, vol. 11, no. 23–581, p. 81, 2010.
- [24] O. Chapelle and L. Li, "An empirical evaluation of thompson sampling," *Advances in neural information processing systems*, vol. 24, 2011.
- [25] A. Kipf, T. Kipf, B. Radke, V. Leis, P. Boncz, and A. Kemper, "Learned cardinalities: Estimating correlated joins with deep learning," *arXiv preprint arXiv:1809.00677*, 2018.
- [26] J. Sun, G. Li, and N. Tang, "Learned cardinality estimation for similarity queries," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 1745–1757.
- [27] J. Sun, J. Zhang, Z. Sun, G. Li, and N. Tang, "Learned cardinality estimation: A design space exploration and a comparative evaluation," *Proceedings of the VLDB Endowment*, vol. 15, no. 1, pp. 85–97, Nov. 2021.
- [28] C. Wu, A. Jindal, S. Amizadeh, H. Patel, W. Le, S. Qiao, and S. Rao, "Towards a learning optimizer for shared clouds," *Proceedings of the VLDB Endowment*, vol. 12, no. 3, pp. 210–222, 2018.
- [29] Z. Yang, E. Liang, A. Kamsetty, C. Wu, Y. Duan, X. Chen, P. Abbeel, J. M. Hellerstein, S. Krishnan, and I. Stoica, "Deep unsupervised cardinality estimation," *Proc. VLDB Endow.*, vol. 13, no. 3, p. 279–292, Nov. 2019.
- [30] R. Marcus and O. Papaemmanouil, "Plan-structured deep neural network models for query performance prediction," *Proc. VLDB Endow.*, vol. 12, no. 11, p. 1733–1746, Jul. 2019.
- [31] J. Sun and G. Li, "An end-to-end learning-based cost estimator," vol. 13, no. 3, 2019.
- [32] F. Liu and S. Blanas, "Forecasting the cost of processing multi-join queries via hashing for main-memory databases," in *Proceedings of the Sixth ACM Symposium on Cloud Computing*, 2015, pp. 153–166.
- [33] W. Wu, Y. Chi, S. Zhu, J. Tatemura, H. Hacigümüş, and J. F. Naughton, "Predicting query execution time: Are optimizer cost models really unusable?" in *2013 IEEE 29th International Conference on Data Engineering (ICDE)*. IEEE, 2013, pp. 1081–1092.
- [34] Y. Chen, M. A. Hoque, P. Xu, J. Lu, and S. Tarkoma, "Simcost: cost-effective resource provision prediction and recommendation for spark workloads," *Distributed and Parallel Databases*, vol. 42, no. 1, pp. 73–102, 2024.
- [35] I. Trummer, J. Wang, Z. Wei, D. Maram, S. Moseley, S. Jo, J. Antonakakis, and A. Rayabari, "Skinnerdb: Regret-bounded query evaluation via reinforcement learning," *ACM Transactions on Database Systems (TODS)*, vol. 46, no. 3, pp. 1–45, 2021.
- [36] X. Yu, G. Li, C. Chai, and N. Tang, "Reinforcement learning with tree-lstm for join order selection," in *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 2020, pp. 1297–1308.
- [37] M. Stillger, G. M. Lohman, V. Markl, and M. Kandil, "Leo-db2's learning optimizer," in *VLDB*, vol. 1, 2001, pp. 19–28.
- [38] S. Krishnan, Z. Yang, K. Goldberg, J. Hellerstein, and I. Stoica, "Learning to optimize join queries with deep reinforcement learning," *arXiv preprint arXiv:1808.03196*, 2018.
- [39] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, and N. Tatbul, "Neo: A learned query optimizer," *Proc. VLDB Endow.*, vol. 12, no. 11, pp. 1705–1718, 2019.
- [40] Z. Yang, W.-L. Chiang, S. Luan, G. Mittal, M. Luo, and I. Stoica, "Balsa: Learning a query optimizer without expert demonstrations," in *Proceedings of the 2022 International Conference on Management of Data*, 2022, pp. 931–944.
- [41] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, and T. Kraska, "Bao: Making learned query optimization practical," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 1275–1288.
- [42] G. M. Lohman, "Grammar-like functional rules for representing query optimization alternatives," *ACM SIGMOD Record*, vol. 17, no. 3, pp. 18–27, 1988.
- [43] P. Dintyala, A. Narechania, and J. Arulraj, "Sqlcheck: Automated detection and diagnosis of sql anti-patterns," in *Proceedings of the 2020 ACM SIGMOD international conference on management of data*, 2020, pp. 2331–2345.
- [44] Q. Bai, S. Alsudais, and C. Li, "Demo of querybooster: Supporting middleware-based sql query rewriting as a service," *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 4038–4041, 2023.
- [45] J. Liu and B. Mozafari, "Query rewriting via large language models," *arXiv preprint arXiv:2403.09060*, 2024.
- [46] Z. Sun, X. Zhou, G. Li, X. Yu, J. Feng, and Y. Zhang, "R-bot: A systematic large language model system for robust query rewriting," *Proceedings of the VLDB Endowment*, vol. 17, no. 11, pp. 3535–3548, 2024.

- [47] Y. Song, H. Yan, J. Lao, Y. Wang, Y. Li, Y. Zhou, J. Wang, and M. Tang, “Quite: A query rewrite system beyond rules with llm agents,” *arXiv preprint arXiv:2506.07675*, 2025.
- [48] Oracle database. [Online]. Available: <https://www.oracle.com/>
- [49] Ibm db2. [Online]. Available: <https://www.ibm.com/products/db2>
- [50] Y. Chen, A. Pan, H. Lei, A. Ye, S. Han, Y. Tang, W. Lu, Y. Chai, F. Zhang, and X. Du, “TDSQL: tencent distributed database system,” *Proc. VLDB Endow.*, vol. 17, no. 12, pp. 3869–3882, 2024.
- [51] Microsoft sql server. [Online]. Available: <https://www.microsoft.com/sql-server>
- [52] Amazon rds. [Online]. Available: <https://aws.amazon.com/rds/>
- [53] P. Negi, M. Interlandi, R. Marcus, M. Alizadeh, T. Kraska, M. Friedman, and A. Jindal, “Steering query optimizers: A practical take on big data workloads,” in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 2557–2569.
- [54] X. Zhou, G. Li, C. Chai, and J. Feng, “A learned query rewrite system using monte carlo tree search,” *Proceedings of the VLDB Endowment*, vol. 15, no. 1, pp. 46–58, 2021.
- [55] Q. Zhou, J. Arulraj, S. Navathe, W. Harris, and J. Wu, “Sia: Optimizing queries using learned predicates,” in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 2169–2181.
- [56] Y. R. Wang, M. Abo Khamis, H. Q. Ngo, R. Pichler, and D. Suciu, “Optimizing recursive queries with program synthesis,” in *Proceedings of the 2022 International Conference on Management of Data*, 2022, pp. 79–93.
- [57] R. Dong, J. Liu, Y. Zhu, C. Yan, B. Mozafari, and X. Wang, “Slabcity: Whole-query optimization using program synthesis,” *Proceedings of the VLDB Endowment*, vol. 16, no. 11, pp. 3151–3164, 2023.
- [58] OpenAI, “Chatgpt,” n.d., accessed: 2025-05-30. [Online]. Available: <https://chatgpt.com/>
- [59] Y. Zhang, Y. Chen, Y. Yu, J. Huang, R. Mao, A. Pant, L. Zheng, and J. Qin, “Efficient query rewrite rule discovery via standardized enumeration and learning-to-rank(extend),” *arXiv preprint arXiv:2603.04169*, 2026.
- [60] L. Mou, G. Li, L. Zhang, T. Wang, and Z. Jin, “Convolutional neural networks over tree structures for programming language processing,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 30, no. 1, 2016.
- [61] R. Zhu, W. Chen, B. Ding, X. Chen, A. Pfadler, Z. Wu, and J. Zhou, “Lero: A learning-to-rank query optimizer,” *Proc. VLDB Endow.*, vol. 16, no. 6, p. 1466–1479, Feb. 2023.
- [62] J. H. Friedman, “Greedy function approximation: a gradient boosting machine,” *Annals of statistics*, pp. 1189–1232, 2001.
- [63] Microsoft Corporation, “Microsoft SQL Server 2019,” Software, 2019, available at <https://www.microsoft.com/en-us/sql-server/sql-server-2019>.
- [64] Shenzhen Institute of Computing Sciences, “YashanDB: A database management system,” <https://yashandb.com>, 2025.